

Answers

1. Factors that affect reliability of a programming language can be listed as follows: simplicity, orthogonality, control structures, data types and structures, syntax, abstraction, expressivity, type checking, and restricted aliasing. We can analyze these factors for the C programming language as follows:

Simplicity: C is not a simple language. For example, there are 4 ways to increment an integer variable in C language

```
count++;
```

```
++count;
```

```
count += 1;
```

```
count = count + 1;
```

Orthogonality: C is not an orthogonal language. As an example, a C function can not return an array variable.

Control structures: C supports loop structures like “for”, “while”; and selection structures like “if”, “switch”.

Data types and structures: C doesn't support all kind of data types. For example C doesn't support Boolean type.

Syntax: Syntax of the C language affects readability and writability in a negative way. For example in C, blocks are written between { and }. In case of nested blocks, it is not easy to see where a block starts and ends.

Abstraction: C supports for abstraction. As an example, in C, we can define and call functions.

Expressivity: In C, we have powerful operators. For example we can increment an integer variable count by simply writing count++ or ++count.

Type checking: In C, some type errors can be detected during the compile time. However, C makes some automatic type conversion between convertible data types, and this property may cause some data type errors and the computed results may be erroneous.

Exception handling: C does not support exception handling.

Restricted Aliasing: By using pointers we can define aliases in C language. So aliasing is not restricted in C.

According to the above factors, we can say that C is not a reliable programming language.

2. In Java language, an identifier starts with a letter, a dollar sign, or an underscore; and continues with any combination of letter, dollar sign, underscore, and digit.

$\langle \text{identifier} \rangle \rightarrow \langle \text{letter} \rangle \langle \text{characters} \rangle \mid _ \langle \text{characters} \rangle \mid \$ \langle \text{characters} \rangle$

$\langle \text{characters} \rangle \rightarrow \epsilon$

$\mid \langle \text{letter} \rangle \langle \text{characters} \rangle$

$\mid \langle \text{digit} \rangle \langle \text{characters} \rangle$

$\mid _ \langle \text{characters} \rangle$

$\mid \$ \langle \text{characters} \rangle$

$\langle \text{letter} \rangle \rightarrow \langle \text{small_letter} \rangle \mid \langle \text{capital_letter} \rangle$

$\langle \text{small_letter} \rangle \rightarrow \text{a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z}$

$\langle \text{capital_letter} \rangle \rightarrow \text{A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z}$

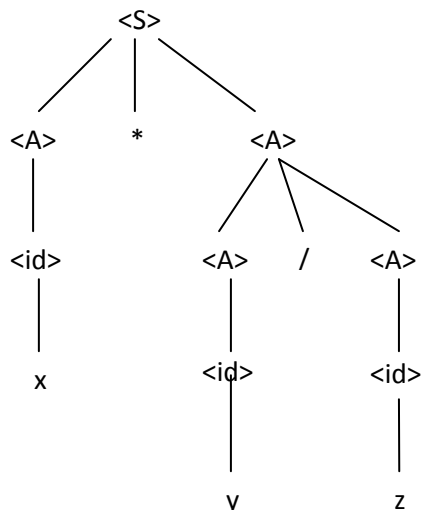
$\langle \text{digit} \rangle \rightarrow 0|1|2|3|4|5|6|7|8|9$

We can rewrite the top two rules as follows:

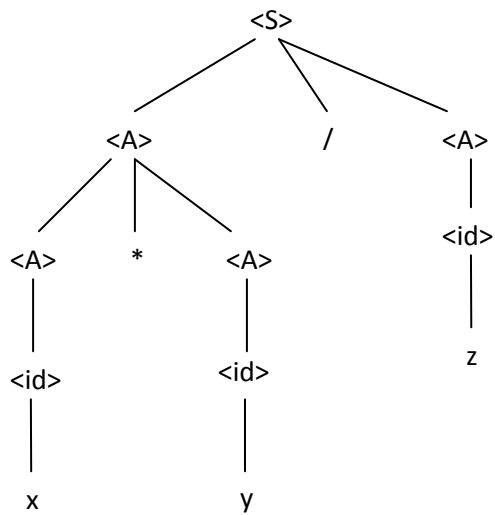
$\langle \text{identifier} \rangle \rightarrow (\langle \text{letter} \rangle \mid _ \mid \$) \{ (\langle \text{letter} \rangle \mid _ \mid \$ \mid \langle \text{digit} \rangle) \}$

3. According to the grammar given in the question, the strings that are generated by the grammar have the form $a^n b^m$ where $n \geq 1$ and $m \geq 1$.

4. The string $x * y / z$ can be generated by the grammar given in this question. For this string, we can generate 2 parse trees as shown below. Since we have 2 parse trees and each parse tree computes $x * y / z$ in a different way, we can say that the grammar given in this question is ambiguous.



According to this parse tree, computation is made as $x * (y / z)$.



According to this parse tree, computation is made as $(x * y) / z$.